

To meet the requirements of a **Real-Time Online Auction System** with concurrency support, we'll use multiple threads to simulate concurrent auctions, with each auction handled by a separate thread. `std::mutex` and `std::lock_guard` will be used to protect shared bid data, ensuring that only one thread can modify the bid data at a time. We'll also use `std::condition_variable` to signal when an auction has ended, allowing other threads to proceed based on that event.

Here's how the solution might look:

```
#include <iostream>
#include <thread>
#include <mutex>
#include <condition_variable>
#include <map>
#include <string>
#include <chrono>
#include <vector>
#include <atomic>

class Auction {
public:
    Auction(std::string item, int duration_seconds)
        : item_name(std::move(item)), duration(duration_seconds), highest_bid(0),
        auction_ended(false) {}

    // Function to place a bid on the item
    void placeBid(int bid, const std::string& bidder) {
        std::lock_guard<std::mutex> lock(bid_mutex);
        if (auction_ended) {
            std::cout << "Auction for " << item_name << " has ended. No more bids allowed.\n";
            return;
        }
        if (bid > highest_bid) {
            highest_bid = bid;
            highest_bidder = bidder;
            std::cout << "New highest bid for " << item_name << ": " << bid << " by " << bidder <<
std::endl;
        } else {
            std::cout << "Bid of " << bid << " by " << bidder << " was too low for " << item_name <<
"\n";
        }
    }
};
```

```

}

// Function to run the auction with a time limit
void startAuction() {
    std::cout << "Starting auction for " << item_name << " lasting " << duration << "
seconds.\n";
    auto start_time = std::chrono::steady_clock::now();

    // Auction loop
    while (std::chrono::steady_clock::now() - start_time < std::chrono::seconds(duration)) {
        std::this_thread::sleep_for(std::chrono::milliseconds(100)); // Simulate time passing
    }

    // End the auction
    {
        std::lock_guard<std::mutex> lock(bid_mutex);
        auction_ended = true;
    }
    cv.notify_all(); // Notify all waiting threads that the auction has ended
    std::cout << "Auction for " << item_name << " has ended.\n";
}

// Function to get the final result of the auction
void waitForAuctionEnd() {
    std::unique_lock<std::mutex> lock(bid_mutex);
    cv.wait(lock, [this] { return auction_ended; });

    if (highest_bid > 0) {
        std::cout << "Auction result for " << item_name << ": Sold to " << highest_bidder
        << " for " << highest_bid << std::endl;
    } else {
        std::cout << "Auction result for " << item_name << ": No bids received.\n";
    }
}

private:
    std::string item_name;
    int duration;
    int highest_bid;
    std::string highest_bidder;
    bool auction_ended;
    std::mutex bid_mutex;
    std::condition_variable cv;
};

```

```

// Simulate bidding by different users
void simulateBidding(Auction& auction, const std::string& bidder, int max_bid) {
    for (int bid = 10; bid <= max_bid; bid += 10) {
        auction.placeBid(bid, bidder);
        std::this_thread::sleep_for(std::chrono::milliseconds(150)); // Simulate time between bids
    }
}

int main() {
    Auction auction1("Painting", 5); // 5-second auction
    Auction auction2("Antique Vase", 7); // 7-second auction

    // Start auctions in separate threads
    std::thread auction_thread1(&Auction::startAuction, &auction1);
    std::thread auction_thread2(&Auction::startAuction, &auction2);

    // Simulate bidding in separate threads for each auction
    std::thread bidder1(simulateBidding, std::ref(auction1), "Alice", 100);
    std::thread bidder2(simulateBidding, std::ref(auction1), "Bob", 80);
    std::thread bidder3(simulateBidding, std::ref(auction2), "Charlie", 90);
    std::thread bidder4(simulateBidding, std::ref(auction2), "Dave", 120);

    // Wait for auctions to end and display results
    std::thread result_thread1(&Auction::waitForAuctionEnd, &auction1);
    std::thread result_thread2(&Auction::waitForAuctionEnd, &auction2);

    // Join all threads
    auction_thread1.join();
    auction_thread2.join();
    bidder1.join();
    bidder2.join();
    bidder3.join();
    bidder4.join();
    result_thread1.join();
    result_thread2.join();

    return 0;
}

```

Explanation of the Code

1. Auction Class:

- `Auction` represents an individual auction with an item name, auction duration, highest bid, highest bidder, and a flag `auction_ended` to indicate if the auction has finished.
- The `bid_mutex` protects shared data (`highest_bid`, `highest_bidder`, and `auction_ended`), while the `cv` condition variable is used to notify when the auction ends.

2. `placeBid` Method:

- This method allows a user to place a bid on the item. It uses `std::lock_guard` to ensure thread-safe access to bid-related data.
- If the auction has ended (`auction_ended == true`), further bids are rejected. If the bid is higher than the current highest bid, it is recorded as the new highest bid.

3. `startAuction` Method:

- This method simulates running an auction with a time limit. It sleeps periodically to simulate the passage of time.
- Once the auction duration has passed, it sets `auction_ended` to `true` and calls `cv.notify_all()` to inform all threads waiting on `cv` that the auction is complete.

4. `waitForAuctionEnd` Method:

- This method waits for the auction to end using a condition variable. Once notified, it checks the highest bid and displays the final auction result.

5. Main Function:

- Two `Auction` objects are created, each with a different item and duration.
- Each auction is started in a separate thread (`auction_thread1`, `auction_thread2`).
- Multiple bidders are simulated with threads (`bidder1`, `bidder2`, etc.) that place bids at intervals.
- Once the auction has ended, results are fetched by `result_thread1` and `result_thread2`, which call `waitForAuctionEnd` to display the auction outcome.

Key Points

- **Concurrency:**
 - Each auction and each bidder is handled in separate threads, simulating concurrent bidding across multiple auctions.
- **Mutex and Condition Variable Usage:**
 - A `std::mutex` (`bid_mutex`) protects bid data within the auction, and `std::lock_guard` ensures thread-safe updates.
 - A `std::condition_variable` (`cv`) is used to notify when an auction ends, allowing any threads waiting for the end of the auction to proceed.
- **Result Synchronization:**

- After the auction time ends, all bidders stop bidding, and results are displayed in separate result threads.

This code demonstrates how to handle concurrent bidding in a simulated online auction system using C++ concurrency primitives, ensuring that bid data is safely updated and that auction end notifications are handled appropriately.